Hamburg University of Technology
Institute of Mathematics
Chair Numerical Mathematics

Jens-Peter M. Zemke
Finn Sommer
Lars Stietz

Advanced Machine Learning
Winter Semester 2025/2026
Exercise Sheet 3

Week 4, 03.11.2025 – 07.11.2025

Remark: From now on we develop our own simple machine learning library. In order to use this library it has to be registered with `pip`. You can use the script `install_amllib.py` for this. This script registers the library as editable library, hence all changes are directly accessible. The script `uninstall_amllib.py` reverts the installation. In the first skeleton for our `amllib` you can find the three directories `activations`, `networks`, and `utils`. Some OS refuse to install additional packages using `pip`, in this case you should set up and activate a virtual environment (if you have not done this yet) via

```
python -m venv /path/to/new/virtual/environment/aml
source /path/to/new/virtual/environment/aml/bin/activate
```

as also explained in the Stud.IP wiki. In this virtual environment you can install your packages via `pip`. To deactivate your virtual environment simply type `deactivate`.

Homework:

Exercise 1: (3+1 points) level •○○ relevance •••

a) Add as many other activation functions to the `activations` package in the skeleton of `amllib` as you like, e.g., those mentioned in the lecture[1]. Each activation function should be implemented as class and inherit from the base class `Activation`, see `activation.py` in the `activations` package, which requires the implementation of
   - `__call__(self, x)`: apply the activation function componentwise,
   - `derive(self, x)`: apply the derivative of the activation function componentwise,
   - `feedforward(self, x)`: apply the activation function componentwise and cache data for the backpropagation,
   - `backprop(self, delta)`: use the data cached by the `feedforward` method to apply the derivative of the activation function componentwise to the last input of `feedforward`, multiply the result componentwise with `delta`.

   Hint: the corresponding `ReLU` implementation derived from the first exercise sheet is already contained in the `activations` package. The use of this class is exemplified in `tests/test_activations.py`.

b) Do you have ideas for other activation functions than the ones mentioned in the lecture? You could search the internet or take a look at those gathered in Wikipedia.

[1] See the lecture notes, Section 1.3

Exercise 2: (2+2 points) level ●●● relevance ●○○
Read the Wikipedia entry[2]

https://en.wikipedia.org/wiki/Kolmogorov%E2%80%93Arnold_representation_theorem

and interpret the Kolmogorov–Arnold representation theorem (a) and the variant by Sprecher (b) each as a special type of feedforward neural network.

Exercises (in class):

Exercise 3: (3 points) level ●●○ relevance ●○○
Prove that any continuous function

$$f : [a, b] \to \mathbb{R}$$

can be approximated arbitrarily well (in some sense, e.g., rather informally) by a neural network $N : \mathbb{R} \to \mathbb{R}$ with a single hidden layer comprising n neurons for some $n \in \mathbb{N}$ based on the ReLU activation in the hidden layer and any or no activation on output.

Hint: you can try to express the linear interpolant of f through the n equidistant points

$$x_1 = a = a + 0h, \quad x_2 = a + h = a + 1h, \quad \ldots, \quad x_n = a + (n-1)h = b$$

as linear combination of the functions $\mathsf{ReLU}(x + b_i)$, where $b_i = -x_{i-1}$, $i = 1, \ldots, n$, $x_0 = a - h$.

Exercise 4: (3+3 points) level ●●● relevance ●●●

a) Use the skeleton in `networks/feedforward.py` to implement backpropagation[3] for our class `FeedforwardNet`.
   (i) Implement a method `feedforward(self, x)` that evaluates the network like it is implemented in `__call__(self, x)`, but also caches the activations and affine combinations for the backpropagation.
   (ii) Add a method `backprop(self, x, y)` that computes and returns the derivatives of the cost function with respect to the weights and the biases for the training data `x` and `y` corresponding to one minibatch.
   (iii) Add a method `update(self)` that uses SGD to update [4] the network parameters after the backpropagation.
   (iv) Add a method `train(self, x, y, batch_size=1, epochs=10)` that uses repeated calls to `feedforward()`, `backprop()`, and `update()` to carry out `epochs` epochs of SGD with batch size `batch_size` for the training data `x` and `y` corresponding to the whole training set.

b) Test your implementation with the MNIST dataset. To load and preprocess the MNIST dataset, the package `utils` provides the module `mnist`. This module provides the functions
   - `load_data()`: load the MNIST dataset (the pixel values are scaled to $[0, 1]$),

[2] You might also want to consider the lecture notes, p. 39+40.
[3] See the lecture notes, Section 2.2
[4] Use `p -= a*dp` instead of `p = p - a*dp` to update a parameter `p` within the update method since the update is discarded otherwise due to the argument handling in Python.

- `encode_labels(labels)`: one-hot encode MNIST labels,
- `flatten_input(input)`: flatten MNIST images from shape `(28,28)` to shape `(784,)`.

Note that you have to transpose the data loaded and processed by this module in order to use it with our `FeedforwardNet` class. In the skeleton you can find the file `tests/test_feedforward.py` that already includes the call to `utils/mnist.py`.